1 Introduction

A compiler is a complex program that translates source code written in one language (the source language) into another language (the target language), often machine code. This translation process is broken into several distinct phases. This report focuses on the first three "front-end" phases.

1.1 Lexical Analysis

Also known as scanning or tokenization, this is the first phase of the compiler. The lexical analyzer reads the source code as a stream of characters and groups them into meaningful sequences called **lexemes**. For each lexeme, the analyzer produces a **token** as output, typically in the form `(token-name, attribute-value)`.

- **Purpose:** To simplify the job of the parser by abstracting the raw character stream into a sequence of classified tokens (e.g., keyword, identifier, number, operator).
- **Example:** The character sequence `var = 5;` would be broken into tokens like `(ID, "var")`, `(ASSIGN_OP, "=")`, `(NUMBER, "5")`, and `(SEMICOLON, ";")`.
- **Tool: Flex** (Fast Lexical Analyzer Generator) is a tool that generates lexical analyzers from a set of regular expressions and C code actions.

1.2 Syntax Analysis

Also known as parsing, this is the second phase. The parser takes the list of tokens generated by the lexical analyzer and attempts to build a hierarchical structure (like a **parse tree** or **abstract syntax tree (AST)**) according to a formal grammar that defines the language's syntax.

- **Purpose:** To verify that the sequence of tokens forms a syntactically valid "sentence" in the language. It catches grammatical errors.
- **Example:** Given the tokens for `var = 5;`, the parser would confirm it matches a rule like `assignment_statement -> ID ASSIGN_OP expression SEMICOLON`. It would reject `var 5 = ;` as a syntax error.
- **Tool: Bison** is a parser generator (a YACC-compatible tool) that generates a parser from a context-free grammar.

1.3 Semantic Analysis

This phase checks the source code for semantic (meaning-related) errors and gathers type information for the subsequent code-generation phase. It operates on the parse tree created by the syntax analyzer.

- **Purpose:** To enforce rules that are not captured by the grammar alone. This includes:
  - **Type Checking:** Ensuring that operators are applied to compatible operands (e.g., rejecting `"hello" + 5` if the language doesn't support it).
  - **Declaration Check:** Verifying that every variable is declared before it is used.
  - **Scope Check:** Resolving names to their correct definitions based on scopes.
- **Tool:** Semantic analysis is typically implemented as C/C++ code **actions embedded within the Bison grammar rules**. It often uses a **Symbol Table** to store information about identifiers like variables and functions.

2 Problem 1

2.1 Problem Statement

Consider the given statements:

1. RUET ##CSE_2018##
2. KUET ##EEE_2018$#
3. BUET $#ECE 2018##

a) Show a flex file which can tokenize given statements.
b) Show a bison file which can parse given statements.

2.2 Solution: Lexical Analysis (l.l)

The Flex file defines regular expressions to match the university names and the complex department/year strings as whole tokens.

```
%option noyywrap

%{
 #include <stdio.h>
  #include <string.h>
  #include <stdlib.h>
  #include "p.tab.h"
%}

uni (RUET|KUET|BUET)
dept1 (##CSE_)
dept2 (##EEE_)
dept3 (\$#ECE[ ])
s_char [#$]
digit [0-9]
year ({digit}{4})
ws [ \t\n]+
f_dept1 {dept1}{year}##
f_dept2 {dept2}{year}\$#
f_dept3 {dept3}{year}##

%%

{ws} {}
{uni} {printf("%s \n", yytext); return UNIV;}
{f_dept1} {printf("%s \n", yytext); return F_DEPT;}
{f_dept2} {printf("%s \n", yytext); return F_DEPT;}
{f_dept3} {printf("%s \n", yytext); return F_DEPT;}

%%
```

Listing 1: l.l (Flex File)

2.3 Solution: Syntax Analysis (p.y)

The Bison file defines a simple grammar: a valid statement (`s1`) is a `UNIV` token followed by an `F_DEPT` token. The program (`s`) can be one or more such statements.

```
%{
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
void yyerror(const char *s);
int yylex();
```

```
extern int yylineno;
%}

%token UNIV F_DEPT
%start s

%%
s: s1 s
 | s1
 ;

s1: UNIV F_DEPT
  ;

%%

int main(void){
  printf("Starting Parsing:\n");
  yyparse();
  printf("Parsing completed.\n");
  return 0;
}

void yyerror(const char *s)
{
    fprintf(stderr, "Error: %s at line %d\n", s, yylineno);
    exit(1);
}
```

Listing 2: p.y (Bison File)

2.4 Input and Output

The parser successfully processes the input file, matching three `s1` rules. The `printf` statements in the lexer show the tokens being identified.

2.4.1 Input (Input.txt)

```
RUET ##CSE_2018##
KUET ##EEE_2018$#
BUET $#ECE 2018##
```

2.4.2 Output (Output.txt)

```
Starting Parsing:
RUET
##CSE_2018##
KUET
##EEE_2018$#
BUET
$#ECE 2018##
Parsing completed.
```

3 Problem 2

3.1 Problem Statement

Consider the following code snippet:

```
def var as INTEGER = 0;

 do
    var = var++
 loop until var < 10.0;
```

a) Perform Lexical Analysis on the given code snippet.
b) Perform Syntax Analysis on the given code snippet.
c) Perform Semantic Analysis on the given code snippet.

3.2 Solution: Lexical Analysis (lex.l)

The Flex file tokenizes the input, identifying keywords (`def`, `as`, `INTEGER`), identifiers (`var`), numbers (`0`), floats (`10.0`), and operators.

```
%option noyywrap

%{
 #include <stdio.h>
 #include <string.h>
 #include <stdlib.h>
 #include "p.tab.h"
%}

alpha [A-Za-z]
digit [0-9]
number {digit}+
float {digit}+\.{digit}+
ws [ \t\n]+
identifier {alpha}({alpha}|{digit})*

%%

{ws}                    { /* ignore whitespace */ }
"def"                   { yylval.str = strdup(yytext); return DEF; }
"as"                    { yylval.str = strdup(yytext); return AS; }
"INTEGER"               { yylval.str = strdup(yytext); return INTEGER; }
"do"                    { yylval.str = strdup(yytext); return DO; }
"<"                     { yylval.str = strdup(yytext); return LT; }
"loop"                  { yylval.str = strdup(yytext); return LOOP; }
"until"                 { yylval.str = strdup(yytext); return UNTIL; }
"++"                    { yylval.str = strdup(yytext); return INC; }
"="                     { yylval.str = strdup(yytext); return EQUAL; }
";"                     { yylval.str = strdup(yytext); return SEMICOLON; }
{number}                { yylval.str = strdup(yytext); return NUMBER; }
{float}                 { yylval.str = strdup(yytext); return FLOAT; }
{identifier}            { yylval.str = strdup(yytext); return ID; }

%%
```

Listing 3: lex.l (Flex File)

3.3 Solution: Syntax and Semantic Analysis (p.y)

This Bison file defines the grammar (Syntax Analysis) and embeds C++ code actions (Semantic Analysis) to be executed when a rule is matched. Note the use of C++ `string` and a conceptual `symbol_table` object.

```
%{
#include<stdio.h>
#include<stdlib.h>
```

```
#include<string.h>
#include "semtab.h" // Assumed to contain SymbolTable class
void yyerror(const char *s);
int yylex();
extern int yylineno;
extern char* yytext;
%}

%union {
    char* str;
}

%token <str> DEF AS INTEGER DO LT LOOP UNTIL INC EQUAL SEMICOLON NUMBER FLOAT ID
%start program

%%

program: declaration loop_stmt
       | declaration
       | loop_stmt
       ;

declaration: DEF ID AS INTEGER EQUAL NUMBER SEMICOLON
           {
               printf("Semantic Analysis starts:\n");
               printf("Variable declaration: ");
               printf("Variable: %s, Type: INTEGER, Value: %s\n", $2, $6);
               if(symbol_table.lookup($2)) {
                   printf("Semantic Error: Variable '%s' already declared\n", $2);
               } else {
                   symbol_table.insert($2, $4, $6);
                   printf("Variable '%s' added to symbol table\n", $2);
               }
           }
           ;

loop_stmt: DO assignment LOOP UNTIL condition
           {
               printf("Loop statement parsed\n");
           }
           ;

assignment: ID EQUAL ID INC
          {
              printf("Assignment with increment\n");
              printf("Assigning %s = %s++\n", $1, $3);
              if(!symbol_table.lookup($1)) {
                  printf("Semantic Error: Variable '%s' not declared\n", $1);
              }
              if(!symbol_table.lookup($3)) {
                  printf("Semantic Error: Variable '%s' not declared\n", $3);
              }
              if(symbol_table.lookup($1) && symbol_table.lookup($3)) {
                  string type1 = symbol_table.getType($1);
                  string type2 = symbol_table.getType($3);
                  if(type1 != type2) {
                      printf("Semantic Warning: Type mismatch in assignment\n");
                  }
              }
          }
          ;

condition: ID LT FLOAT SEMICOLON
```

```
            {
                printf("Condition check\n");
                printf("Condition: %s < %s\n", $1, $3);
                if(!symbol_table.lookup($1)) {
                    printf("Semantic Error: Variable '%s' not declared\n", $1);
                }
                if(symbol_table.lookup($1)) {
                    string varType = symbol_table.getType($1);
                    if(varType == "INTEGER") {
                        printf("Semantic Warning: Comparing INTEGER with FLOAT\n");
                    }
                }
            }
            ;

%%

int main(void){

  printf("=== PARSING PHASE ===\n");
  yyparse();
  printf("=== PARSING COMPLETED ===\n\n");

  printf("=== SYMBOL TABLE ===\n");
  symbol_table.display();

  return 0;
}

void yyerror(const char *s)
{
    fprintf(stderr, "Syntax Error: %s at line %d\n", s, yylineno);
    exit(1);
}
```

Listing 4: parse.y (Bison File)

3.4 Analysis of Phases in Problem 2

- **Lexical Analysis:** Performed by the Flex file. It correctly breaks the input string `def var as INTEGER = 0; ...` into a stream of tokens: `DEF`, `ID("var")`, `AS`, `INTEGER`, `EQUAL`, `NUMBER("0")`, `SEMICOLON`, `DO`, `ID("var")`, `EQUAL`, `ID("var")`, `INC`, `LOOP`, `UNTIL`, `ID("var")`, `LT`, `FLOAT("10.0")`, `SEMICOLON`.
- **Syntax Analysis:** Performed by the Bison file's grammar rules (`%% ... %%`). The parser successfully matches the token stream to the `program` rule, which is composed of a `declaration` rule followed by a `loop_stmt` rule. The input is syntactically correct.
- **Semantic Analysis:** Performed by the C++ code actions (inside `{...}`) in the Bison file.
    1. **In `declaration` rule:** The action adds `var` to the symbol table with type `INTEGER` and value `0`. This is a declaration check.
    2. **In `assignment` rule:** The actions check if the variables used (`var` on the left and `var` on the right) exist in the symbol table. This is an undeclared variable check.
    3. **In `condition` rule:** This is the key semantic check. The action looks up the type of `var` (which is `INTEGER`) and compares it against the type of the literal `10.0` (which the lexer tokenized as `FLOAT`). Since `INTEGER` and `FLOAT` are being compared, the action correctly prints a **"Semantic Warning: Comparing INTEGER with FLOAT"**.

3.5 Input and Output

The output clearly shows the `printf` statements from the semantic actions executing as the parser validates the syntax. The semantic warning is successfully generated, and the final symbol table contents are displayed.

3.5.1 Input (Input.txt)

```
def var as INTEGER = 0;
 do
    var = var++
 loop until var < 10.0;
```

3.5.2 Output (Output.txt)

```
=== PARSING PHASE ===
Semantic Analysis starts:
Variable declaration: Variable: var, Type: INTEGER, Value: 0
Variable 'var' added to symbol table
Assignment with increment
Assigning var = var++
Condition check
Condition: var < 10.0
Semantic Warning: Comparing INTEGER with FLOAT
Loop statement parsed
=== PARSING COMPLETED ===

=== SYMBOL TABLE ===

Symbol Table
Name         Type         Value
var          INTEGER      0
```

4 Conclusion

This report successfully defined and differentiated the lexical, syntax, and semantic phases of compilation.

- **Lexical Analysis** was demonstrated using **Flex** to convert raw text into a stream of tokens based on regular expressions.
- **Syntax Analysis** was demonstrated using **Bison** to validate the sequence of tokens against a formal context-free grammar.
- **Semantic Analysis** was demonstrated by embedding C++ actions within the Bison grammar. These actions utilized a symbol table to perform crucial checks, such as variable declaration and type compatibility, proving that an input can be syntactically correct but still contain semantic issues (like comparing an integer to a float).

The two problems illustrate how these phases build upon one another, with Flex, Bison, and embedded C/C++ code working together to fully analyze and validate a piece of source code.